

Software System Design

(2025) Architecture

Assignment 1



软件学院

221850037

王朝晖

2025 年 5 月 11 日

Task 1 Quality Attribute Scenarios

随着 AI 的快速发展和在其软件系统中的深度集成，基于 AI Agent 原生应用的架构研究意义重大。我通过场景化分析方法，识别出基于 AI Agent 的 AI 原生软件应用的四大核心质量属性并构建了对应的评估。

我选取**性能 (Performance)**、**安全性 (Security)**、**可扩展性 (Scalability)** 与**维护性 (Maintainability)** 作为关键评估维度。下面我将对这四个质量属性的一般场景和典型具体场景进行描述。

1.1 性能 (Performance)

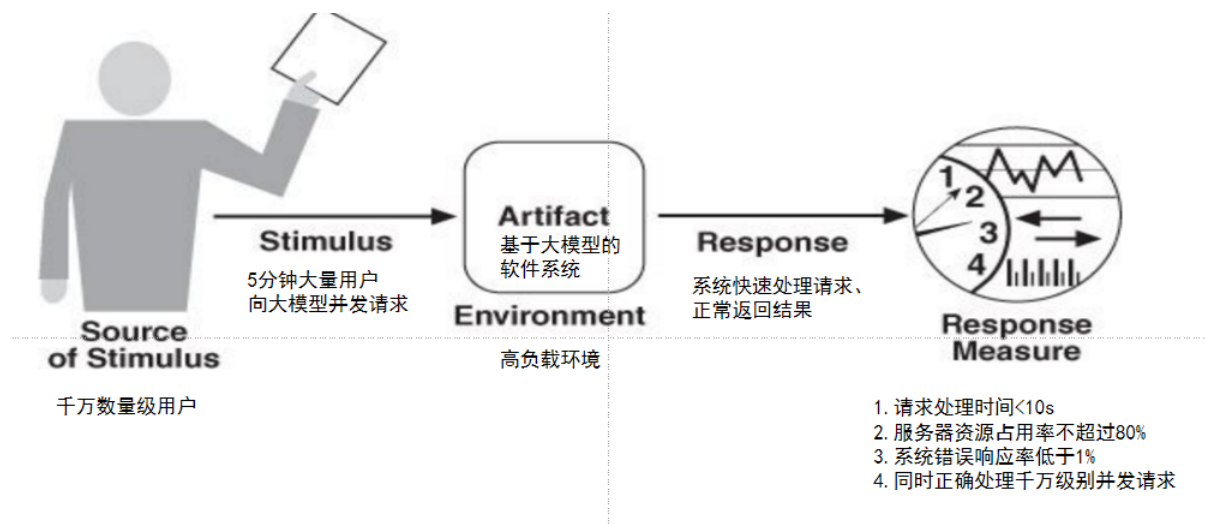
性能是关于时间的质量属性，是软件系统满足时间需求的能力。性能对于基于 AI 的软件系统的重要性不言而喻，它与用户的体验和以及使用效率紧密相关。具体来说，性能常常关于资源使用效率、响应的时间等。

1.1.1 一般场景

性能 (Performance) 的一般场景:

Portion of Scenario	Possible Values
Source	用户或自动化系统
Stimulus	向大语言模型请求处理数据
Artifact	基于大语言模型的软件系统
Environment	正常运行环境、高负载环境
Response	系统处理请求返回结果、系统输出错误到日志
Response Measure	1. 处理时间不超过规定时间 2. 资源占用率低于规定最高值

1.1.2 典型具体场景



1.2 安全性 (Security)

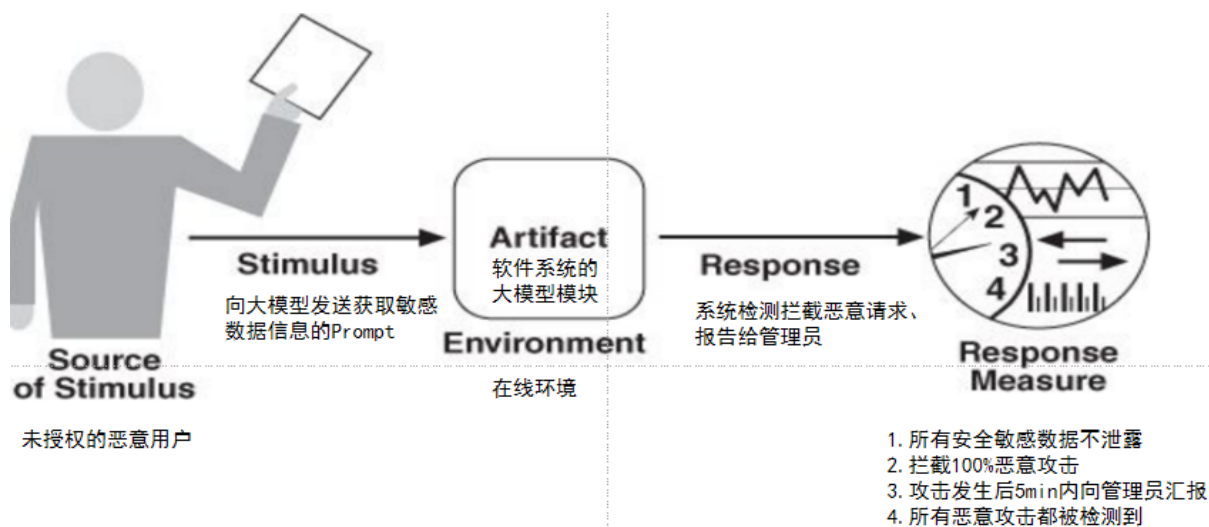
安全性衡量了系统保护数据免受未认证的攻击的能力。对于基于 AI 的软件系统，安全性常常体现在对于用户的敏感隐私数据是否能够保护，是否会涉及恶意代码注入、用户敏感信息泄露等问题。

1.2.1 一般场景

安全性 (Security) 的一般场景:

Portion of Scenario	Possible Values
Source	未授权的用户或自动化系统
Stimulus	尝试注入攻击或者获取隐私数据
Artifact	软件系统以及数据环境
Environment	联网环境
Response	系统识别恶意请求、阻止攻击并报告
Response Measure	1. 未授权的请求被检测 2. 所有敏感数据保证不泄露 3. 记录恶意攻击的详细异常情况并报告管理员

1.2.2 典型具体场景



1.3 可扩展性 (Scalability)

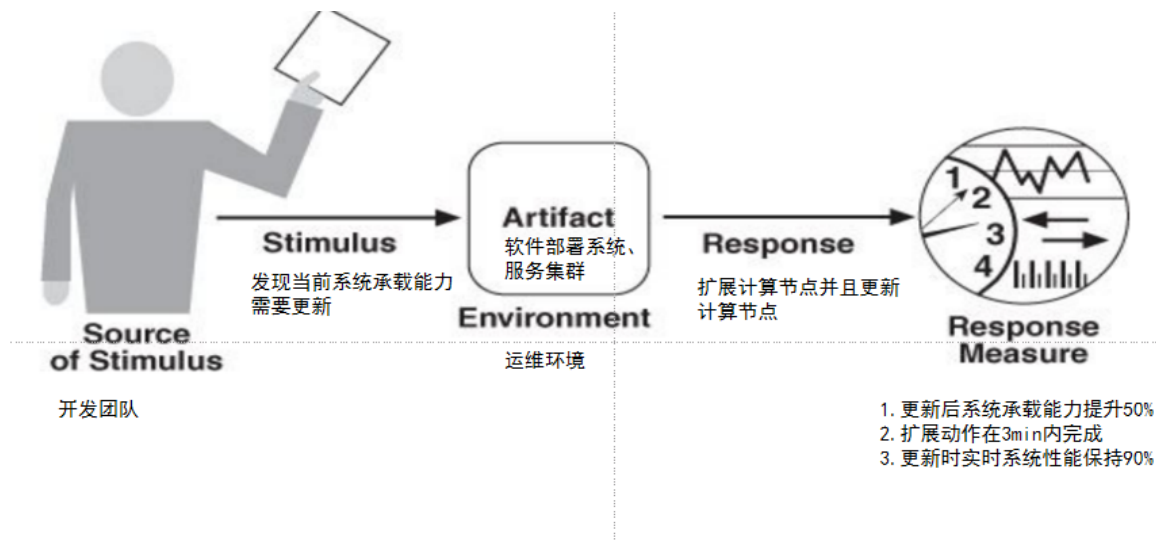
可扩展性指系统在负载增加时维持服务质量的能力。对于基于 AI 的应用，需支持动态扩展计算资源以应对模型推理的突发需求，同时保证分布式架构的协调性。并且对于计算资源的潜在变化，可扩展的架构可以更好适应。这些都是可扩展性对于基于 AI 的应用的重要意义。

1.3.1 一般场景

可扩展性 (Scalability) 的一般场景:

Portion of Scenario	Possible Values
Source	开发团队
Stimulus	当前系统承载能力需要更新
Artifact	模型服务集群、负载均衡组件
Environment	运维环境、云原生环境
Response	扩展计算节点、重新分配模型副本并更新
Response Measure	1. 扩展动作在 5 分钟内完成 2. 扩展后延迟不超过 300ms 3. 资源利用率保持在 70%-90%

1.3.2 典型具体场景



1.4 可维护性 (Maintainability)

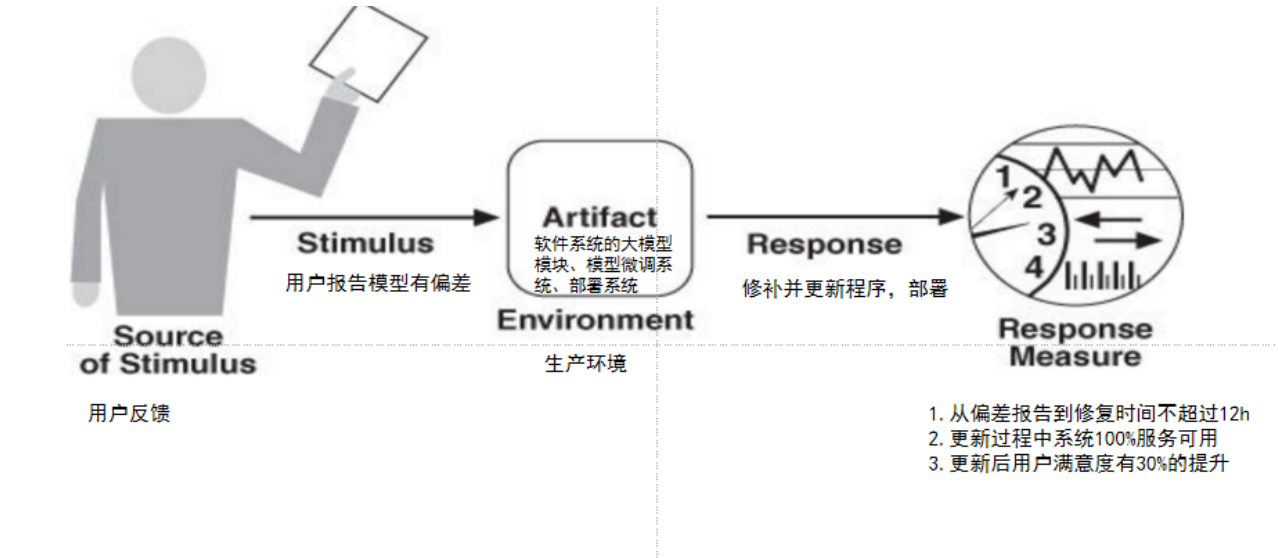
可维护性反映系统支持高效修改更新的能力。AI 系统需快速响应模型迭代、安全补丁和功能增强，同时最小化服务中断。高可维护性对于系统的运行成本和更新成本都有降低作用，并且方便维护软件系统的长期质量。

1.4.1 一般场景

可维护性 (Maintainability) 的一般场景:

Portion of Scenario	Possible Values
Source	开发团队、用户反馈
Stimulus	需要修复模型偏差或升级 API 版本
Artifact	模型微调管道、部署系统
Environment	开发/测试/生产多环境
Response	滚动更新系统、回归测试、部署更新
Response Measure	1. 热更新导致的服务中断 <30 秒 2. 从提交到部署耗时 <2 小时 3. 版本回滚成功率 100%

1.4.2 典型具体场景



Task 2 Tactics

2.1 改进性能 (Performance)

策略	具体战术	影响评估			
		性能	安全性	可扩展性	可维护性
优化系统算法	代码优化减少计算复杂度	显著提升	可能引入漏洞	无直接影响	需持续重构
	采用异步并行计算技术	吞吐量提升	并发风险提升	分布式适配性提升	调试难度提升
优化模型架构	量化压缩模型参数（如FP16→INT8）	推理速度提升 40%	精度损失风险	边缘部署能力提升	需重训练
	实现动态批处理（Dynamic Batching）	GPU 利用率提升	请求隔离性降低	高并发支持提升	调度逻辑复杂度提升

表 1: 性能改进策略评估（新增模型架构优化）

2.2 改进安全性 (Security)

策略	具体战术	影响评估			
		性能	安全性	可扩展性	可维护性
访问控制强化	实施多因素认证 (MFA)	延迟增加	显著提升	无影响	管理成本高
	基于角色的权限管理 (RBAC)	权限校验开销提升	横向渗透风险降低	支持组织架构扩展	策略维护复杂度提升
威胁监控	实时入侵检测系统 (IDS)	CPU 占用提升 3-5%	攻击识别率提升	规则库扩展性提升	需持续更新特征库
	安全日志审计追踪	存储开销提升	事后追溯能力提升	日志分片支持提升	日志分析复杂度提升

表 2: 安全性改进策略评估矩阵

2.3 改进可扩展性 (Scalability)

策略	具体战术	影响评估			
		性能	安全性	可扩展性	可维护性
分布式架构	部署微服务架构	延迟略增 5%	服务隔离性提升	支持横向扩展	部署复杂度提升
	实现数据库分片 (Sharding)	查询效率提升	数据一致性风险	数据容量扩展性提升	分片管理成本增加
缓存优化	引入分布式缓存 (如 Redis)	响应时间降低 50%	缓存一致性风险	高并发支持提升	需监控缓存命中率
	实现客户端缓存 (CDN)	静态资源加载加速	缓存失效风险	全球访问能力提升	需配置缓存策略

表 3: 可扩展性改进策略评估矩阵

2.4 改进可维护性 (Maintainability)

策略	具体战术	影响评估			
		性能	安全性	可扩展性	可维护性
模块化设计	实现组件化开发	模块调用略增开销	接口安全性需关注	模块独立扩展性提升	代码复用性提升
	标准化 API 接口	无明显影响	需校验输入参数	跨系统集成性提升	文档维护成本降低
自动化运维	实施 CI/CD 流水线	部署速度提升	需保护流水线安全	快速迭代能力提升	自动化测试覆盖率提升
	自动化监控与告警	轻微 CPU 占用	异常检测能力提升	监控点扩展性提升	故障定位效率提升

表 4: 可维护性改进策略评估矩阵

Task 3: Alternative Tactics Debate

3.1 心跳检测 (Heartbeat)

Heartbeat 是一种主动检测机制，系统定期向监控节点发送状态更新信号，表明其可用性。监控节点根据信号的接收情况判断系统状态。

3.1.1 好处 (Benefits)

- 1. **自动化检测:** 定时发送心跳，监控系统无需主动发起请求，适合持续运行的服务。
 - (a) 支持指数退避策略 (1s→60s)
 - (b) 典型应用：Kubernetes 节点健康检查
- 2. **单向安全:** 仅需单向通信，减少暴露接口，降低被攻击的风险。
- 3. **业务扩展:** 心跳信号可携带状态信息（如负载、资源使用率），支持额外的业务功能。例如，微软 Azure 用于预测硬件故障。

3.1.2 成本 (Costs)

1. **资源消耗:** 每次心跳发送消耗网络和计算资源，尤其在高频检测时。
2. **配置复杂度:** 需要精确设置心跳间隔，过短浪费资源，过长延迟故障检测。
3. **单点依赖:** 监控节点故障可能导致误判，需额外冗余设计。

3.2 Ping-Echo

Ping-Echo 是一种被动检测机制，监控节点向系统发送请求 (ping)，系统响应 (echo) 以证明其可用性。

3.2.1 好处 (Benefits)

1. **灵活检测:** 监控节点可按需调整检测频率，适应动态负载场景。
 - (a) 腾讯云 CLB 可以根据延迟调整流量
 - (b) 支持动态间隔（高峰期 5s）
2. **双向确认:** 确保双方通信正常，减少误判可能性。
3. **简单实现:** 标准协议（如 ICMP）易于部署，适合多种系统环境。Nginx 配置代码量低，难度低。跨平台兼容性高。

3.2.2 成本 (Costs)

1. **高资源消耗:** 每次检测需双向通信，网络和计算开销是心跳的两倍。
2. **安全风险:** 双向通信暴露更多接口，可能被利用进行拒绝服务攻击。
3. **响应依赖:** 系统高负载时可能延迟响应，导致误判为故障。

3.3 收益与成本比较 (Benefits and Costs Comparison)

策略	具体战术	评估维度			
		资源消耗	安全性	灵活性	检测效率
心跳检测	定期发送状态信号	中等（单向通信）	较高（单向安全）	较低（固定间隔）	高（自动化）
	携带业务状态更新	中等（附加数据）	较高（可加密）	中等（支持扩展）	高（多功能）
Ping-Echo	按需发送 ping 请求	较高（双向通信）	较低（暴露接口）	高（动态调整）	中等（响应依赖）
	使用标准 ICMP 协议	较高（双向开销）	中等（标准协议）	高（易部署）	中等（需确认）

表 5: 心跳与 Ping-Echo 检测策略评估矩阵

3.4 适用场景 (Applicable Scenarios)

3.4.1 心跳检测的适用场景

3.4.1.1 分布式数据库集群

- **场景描述:** 分布式数据库（如 Cassandra）需要持续监控节点状态，确保数据一致性和高可用性。
- **选择原因:** 心跳检测通过节点定期发送状态信号（包括负载和健康信息），支持自动故障检测和数据重新分配。单向通信减少了网络暴露，适合高安全需求的场景。

3.4.1.2 工业物联网设备

- **场景描述:** 工业物联网设备（如传感器）需定期报告状态，资源受限且网络带宽有限。
- **选择原因:** 心跳检测允许设备以低频率发送信号，节省带宽和电池，同时支持状态更新（如温度、故障码），满足自动化监控需求。

3.4.2 Ping-Echo 检测的适用场景

3.4.2.1 云服务负载均衡器

- **场景描述:** 云服务负载均衡器需动态检测后端服务器的可用性，以分配流量。
- **选择原因:** Ping-Echo 允许负载均衡器按需发起检测，灵活调整频率以适应流量波动。双向确认确保服务器响应能力，适合高动态场景。

3.4.2.2 企业网络监控

- **场景描述:** 企业网络需监控大量设备的连通性（如路由器、交换机），环境复杂且设备异构。
- **选择原因:** Ping-Echo 使用标准 ICMP 协议，易于部署到不同设备。灵活的检测频率适应网络规模变化，双向通信确认设备活性。

3.5 决策标准 (Decision Criteria)

选择心跳检测或 Ping-Echo 检测应基于以下标准：

- **安全性需求:** 高安全场景（如数据库、物联网）优先选择心跳检测，因其单向通信减少攻击面；Ping-Echo 适合较低安全需求的网络监控。
- **资源约束:** 资源受限环境（如物联网设备）倾向于心跳检测，减少通信开销；资源充足的云服务可使用 Ping-Echo 以获灵活性。
- **检测灵活性:** 动态负载场景（如负载均衡器）适合 Ping-Echo，因其可按需调整检测频率；固定节奏的系统（如数据库）适合心跳检测。
- **业务功能需求:** 若需附加状态更新（如负载信息），心跳检测更合适；若仅需确认可用性，Ping-Echo 更简单。

心跳检测通过自动化和单向通信优化了安全性和资源效率，适合稳定、高安全场景。Ping-Echo 通过灵活性和双向确认适应动态、异构环境。选择时需权衡系统需求与资源限制，确保检测效率与业务目标一致。